



Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones
Semantics

Module Summary

Programming in Modern C++

Module M21: Inheritance: Part 1: Inheritance Semantics

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Weekly Recap

Module M21

Partha Pratim Das

Weekly Recap

Objectives & Outline

ISA Relationship

Inheritance in C++

Phones

Semantics

Module Summary

- Understood `static` data members and member functions; and their use in modeling, especially Singleton Classes
- Understood `friend` function and `friend` class
- Noted its benefits vis-a-vis potential risk on breaking encapsulation
- Introduced operator overloading for user-defined types
- Outlined semantics for overloading binary and unary operators
- Learnt to build overloaded operators for UDTs using global, member or `friend` functions
- Understood `namespace` as a lexical scoping tool in C++



Module Objectives

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones

Semantics

Module Summary

- Understand ISA Relationship in OOAD and understand how hierarchy can be created in C++ with Inheritance

NPTEL



Module Outline

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones
Semantics

Module Summary

1 Weekly Recap

2 ISA Relationship

3 Inheritance in C++

- Phones
- Semantics

4 Module Summary



ISA Relationship

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones

Semantics

Module Summary

NPTEL

ISA Relationship



ISA Relationship

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones

Semantics

Module Summary

- We often find one object is a *specialization* / *generalization* of another
- OOAD models this using **ISA** relationship
- C++ models **ISA** relationship by *Inheritance* of classes



ISA Relationship

Module M21

Partha Pratim Das

Weekly Recap

Objectives & Outline

ISA Relationship

Inheritance in C++

Phones

Semantics

Module Summary

- **Rose ISA Flower**

- Rose has the properties of Flower – like fragrance, having petals etc.
- Rose has some additional properties – like rosy fragrance
- Rose is a *specialization* of Flower
- Flower is a *generalization* of Rose

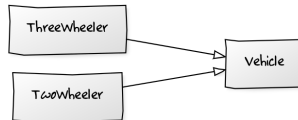
- **Red Rose ISA Rose**

- Red Rose has the properties of Rose – like rosy fragrance etc.
- Red Rose has some additional properties – like it is red
- Red Rose is a *specialization* of Rose

- Rose is a *generalization* of Red Rose



- **TwoWheeler ISA Vehicle; ThreeWheeler ISA Vehicle**



- **Manager ISA Employee**





Inheritance in C++

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones
Semantics

Module Summary

Inheritance in C++



Inheritance in C++: Hierarchy

Module M21

Partha Pratim Das

Weekly Recap

Objectives & Outline

ISA Relationship

Inheritance in C++

Phones
Semantics

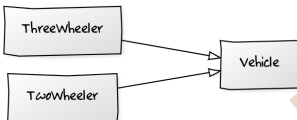
Module Summary

- **Manager ISA Employee** [Single Inheritance]



```
class Employee; // Base Class = Employee
class Manager: public Employee; // Derived Class = Manager; Base Class = Employee
```

- **TwoWheeler ISA Vehicle; ThreeWheeler ISA Vehicle** [Hybrid Inheritance]



```
class Vehicle; // Base Class = Vehicle -- Root
class TwoWheeler: public Vehicle; // Derived Class = TwoWheeler; Base Class = Vehicle
class ThreeWheeler: public Vehicle; // Derived Class = ThreeWheeler; Base Class = Vehicle
```

- **Red Rose ISA Rose ISA Flower** [Multi-Level Inheritance]



```
class Flower; // Base Class = Flower -- Root
class Rose: public Flower; // Derived Class = Rose; Base Class = Flower
class RedRose: public Rose; // Derived Class = RedRose; Base Class = Rose
```



Inheritance in C++: Phones

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones

Semantics

Module Summary

- **Landline Phone**

- Call: By dial / keyboard
- Answer
- Caller ID (with special attached device)

- **Mobile Phone**

- Call: By keyboard – shows number
 - ▷ By Number
 - ▷ By Name
- Answer
- Caller ID
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name

- **Smart Phone**

- Call: By touchscreen – shows number & photo
 - ▷ By Number
 - ▷ By Name
- Answer
- Caller ID
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name
 - ▷ Photo

- There exists a substantial overlap between the functionality of the phones
- A mobile phone is more capable than a land line phone and can perform (almost) all its functions
- A smart phone is more capable than a mobile phone and can perform (almost) all its functions
- These phones belong to a **Specialization / Generalization Hierarchy**



Inheritance in C++: Semantics

Module M21

Partha Pratim Das

Weekly Recap

Objectives & Outline

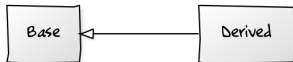
ISA Relationship

Inheritance in C++

Phones Semantics

Module Summary

- Derived ISA Base



```
class Base; // Base Class = Base
class Derived: public Base; // Derived Class = Derived
```

- Use keyword `public` after class name to denote inheritance
- Name of the Base class follow the keyword

Public inheritance means "is-a." Everything that applies to base classes must also apply to derived classes, because every derived class object is a base class object

– *Scott Meyers in Item 32, Effective C++ (3rd. Edition)*



Inheritance in C++: Semantics

Module M21

Partha Pratim Das

Weekly Recap

Objectives & Outline

ISA Relationship

Inheritance in C++

Phones

Semantics

Module Summary

- **Derived ISA Base**
- **Data Members**
 - **Derived** class *inherits* all data members of **Base** class
 - **Derived** class may *add* data members of its own
- **Member Functions**
 - **Derived** class *inherits* all member functions of **Base** class
 - **Derived** class may *override* a member function of **Base** class by *redefining* it with the *same signature*
 - **Derived** class may *overload* a member function of **Base** class by *redefining* it with the *same name*; but *different signature*
 - **Derived** class *may add* new member functions
- **Access Specification**
 - **Derived** class *cannot access private* members of **Base** class
 - **Derived** class *can access protected* members of **Base** class
- **Construction-Destruction**
 - A *constructor* of the **Derived** class *must first* call a *constructor* of the **Base** class to construct the **Base** class instance of the **Derived** class
 - The *destructor* of the **Derived** class *must* call the *destructor* of the **Base** class to destruct the **Base** class instance of the **Derived** class



Module Summary

Module M21

Partha Pratim
Das

Weekly Recap

Objectives &
Outline

ISA Relationship

Inheritance in
C++

Phones
Semantics

Module Summary

- Understood Hierarchy or ISA Relationship in OOAD
- Introduced the Semantics of Inheritance in C++

NPTEL



Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

Programming in Modern C++

Module M22: Inheritance: Part 2: Data Member & Member Function: Override & Overload

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

- Understood Hierarchy or ISA Relationship in OOAD
- Introduced the Semantics of Inheritance in C++

NPTEL



Module Objectives

Module M22

Partha Pratim
Das

Objectives & Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

- Understand how inheritance impacts data members and member functions
- Introduce overriding of member function and its interactions with overloading

NPTEL



Module Outline

Module M22

Partha Pratim
Das

Objectives & Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

1 Inheritance in C++

2 Data Members

- Object Layout

3 Member Functions

- Overrides and Overloads

4 Comparison

5 Module Summary



Inheritance in C++

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

Inheritance in C++



Inheritance in C++: Semantics

Module M22

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

Data Members

Object Layout

Member Functions

Overrides and Overloads

Comparison

Module Summary

- **Derived ISA Base**
- **Data Members**
 - **Derived** class *inherits* all data members of **Base** class
 - **Derived** class may *add* data members of its own
- **Member Functions**
 - **Derived** class *inherits* all member functions of **Base** class
 - **Derived** class may *override* a member function of **Base** class by *redefining* it with the *same signature*
 - **Derived** class may *overload* a member function of **Base** class by *redefining* it with the *same name*; but *different signature*
 - **Derived** class *may add* new member functions
- **Access Specification**
 - **Derived** class *cannot access private* members of **Base** class
 - **Derived** class *can access protected* members of **Base** class
- **Construction-Destruction**
 - A *constructor* of the **Derived** class *must first* call a *constructor* of the **Base** class to construct the **Base** class instance of the **Derived** class
 - The *destructor* of the **Derived** class *must* call the *destructor* of the **Base** class to destruct the **Base** class instance of the **Derived** class



Data Members

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

NPTEL

Data Members



Data Members

Module M22

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

Data Members

Object Layout

Member Functions

Overrides and Overloads

Comparison

Module Summary

- **Derived ISA Base**
- **Data Members**
 - **Derived** class *inherits* all data members of **Base** class
 - **Derived** class may *add* data members of its own
- **Object Layout**
 - **Derived** class *layout* contains an instance of the **Base** class
 - Further, **Derived** class *layout* will have data members of its own
 - C++ does not guarantee the *relative position* of the **Base** class instance and **Derived** class members



Object Layout

Module M22

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

Data Members
Object Layout

Member Functions

Overrides and Overloads

Comparison

Module Summary

```
class B { // Base Class
    int data1B_;
public:
    int data2B_;
    // ...
};
```

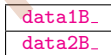
```
class D: public B { // Derived Class
    // Inherits B::data1B_
    // Inherits B::data2B_
    int infoD_; // Adds D::infoD_
public:
    // ...
};
```

```
B b; // Base Class Object
```

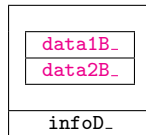
```
D d; // Derived Class Object
```

Object Layout

Object b



Object d



- **d cannot access data1B_** even though is a part of **d**!
- **d can access data2B_**



Member Functions

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

**Member
Functions**

Overrides and
Overloads

Comparison

Module Summary

Member Functions



Member Functions

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members
Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

- **Derived ISA Base**
- **Member Functions**
 - **Derived** class *inherits* all member functions of **Base** class
 - ▷ **Note:** **Derived** class *does not inherit* the **Constructors** and **Destructor** of **Base** class but *must have access to them*
 - **Derived** class may *override* a member function of **Base** class by *redefining* it with the *same signature*
 - **Derived** class may *overload* a member function of **Base** class by *redefining* it with the *same name*; but *different signature*
 - **Derived** class *may add* new member functions
- **Static Member Functions**
 - **Derived** class *does not inherit* the static member functions of **Base** class
- **Friend Functions**
 - **Derived** class *does not inherit* the friend functions of **Base** class



Overrides and Overloads

Module M22

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

Data Members

Object Layout

Member Functions

Overrides and Overloads

Comparison

Module Summary

Inheritance

```
class B { public: // Base Class
    void f(int i);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)

    // Inherits B::g(int)

};
B b;
D d;

b.f(1); // Calls B::f(int)
b.g(2); // Calls B::g(int)

d.f(3); // Calls B::f(int)
d.g(4); // Calls B::g(int)
```

- D::f(int) overrides B::f(int)
- D::f(string&) overloads B::f(int)

Override & Overload

```
class B { public: // Base Class
    void f(int);
    void g(int i);
};
class D: public B { public: // Derived Class
    // Inherits B::f(int)
    void f(int); // Overrides B::f(int)
    void f(string&); // Overloads B::f(int)
    // Inherits B::g(int)
    void h(int i); // Adds D::h(int)
};
B b;
D d;

b.f(1); // Calls B::f(int)
b.g(2); // Calls B::g(int)

d.f(3); // Calls D::f(int)
d.g(4); // Calls B::g(int)

d.f("red"); // Calls D::f(string&)
d.h(5); // Calls D::h(int)
```



Overloading vis-a-vis Overriding

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

Overloading vis-a-vis Overriding



Comparison of Overloading vis-a-vis Overriding

Module M22

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

Data Members

Object Layout

Member Functions

Overrides and Overloads

Comparison

Module Summary

Basis	Function Overloading	Function Overriding
Name of Function Signature Type of Function	• All overloads have the same function name • Function signatures must be different • Can be global, friend, static or non-static member function	• All overrides have the same function name • Function signatures are same • Must be a non-static member function - non-virtual or virtual
Inheritance Polymorphism Scope	• Can happen with or without inheritance • Static (Compile time) • Overloaded functions are in the same scope	• Happens only with inheritance • Static (Compile time) or Dynamic (Runtime) • Functions are in different scopes (base class and derived class)
Purpose	• To have multiple functions with same name that act differently depending on parameters	• To perform additional or different tasks than the base class function
Constructor Destructor Usage	• Constructors can be overloaded • The destructor cannot be overloaded • Can be overloaded multiple times	• Constructors cannot be overridden • The destructor cannot be overridden • Can be overridden once in the derived class



Module Summary

Module M22

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

Data Members

Object Layout

Member
Functions

Overrides and
Overloads

Comparison

Module Summary

- Discussed the effect of inheritance on Data Members and Object Layout
- Discussed the effect of inheritance on Member Functions with special reference to Overriding and Overloading



Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

Programming in Modern C++

Module M23: Inheritance: Part 3: Constructor & Destructor: Object Lifetime

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

- Discussed the effect of inheritance on Data Members and Object Layout
- Discussed the effect of inheritance on Member Functions with special reference to Overriding and Overloading



Module Objectives

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

- Understand `protected` access specifier
- Understand the construction and destruction process on an object hierarchy
- Revisit Object Lifetime for a hierarchy



Module Outline

Module M23

Partha Pratim
Das

Objectives & Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

- 1 Inheritance in C++
- 2 protected Access
 - Streaming
- 3 Constructor & Destructor
- 4 Object Lifetime
- 5 Module Summary



Inheritance in C++

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

Inheritance in C++



Inheritance in C++: Semantics

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

- **Derived ISA Base**
- **Data Members**
 - **Derived** class *inherits* all data members of **Base** class
 - **Derived** class may *add* data members of its own
- **Member Functions**
 - **Derived** class *inherits* all member functions of **Base** class
 - **Derived** class may *override* a member function of **Base** class by *redefining* it with the *same signature*
 - **Derived** class may *overload* a member function of **Base** class by *redefining* it with the *same name*; but *different signature*
 - **Derived** class *may add* new member functions
- **Access Specification**
 - **Derived** class *cannot access private* members of **Base** class
 - **Derived** class *can access protected* members of **Base** class
- **Construction-Destruction**
 - A *constructor* of the **Derived** class *must first* call a *constructor* of the **Base** class to construct the **Base** class instance of the **Derived** class
 - The *destructor* of the **Derived** class *must* call the *destructor* of the **Base** class to destruct the **Base** class instance of the **Derived** class



protected Access

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

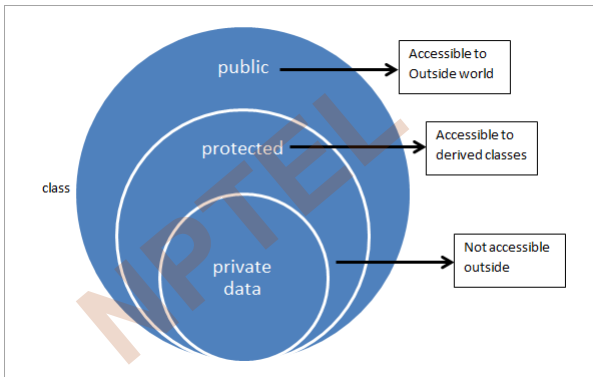
protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary



protected Access



Access Members of Base: protected Access

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

- **Derived ISA Base**
- **Access Specification**
 - **Derived** class *cannot access private* members of **Base** class
 - **Derived** class *can access public* members of **Base** class
- **protected Access Specification**
 - A new **protected** access specification is introduced for **Base** class
 - **Derived** class *can access protected* members of **Base** class
 - **No other class** or **global** function *can access protected* members of **Base** class
 - A **protected** member in **Base** class is like **public** in **Derived** class
 - A **protected** member in **Base** class is like **private** in **other classes** or **global** functions



protected Access

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

private Access

```
class B {  
private: // Inaccessible to child  
        // Inaccessible to others  
    int data_;  
public: // ...  
    void Print() { cout << "B Object: ";  
        cout << data_ << endl;  
    }  
};  
class D: public B { int info_; public: // ...  
    void Print() { cout << "D Object: ";  
        cout << data_ << ", "; // Inaccessible  
        cout << info_ << endl;  
    }  
};  
B b(0);  
D d(1, 2);  
  
b.data_ = 5; // Inaccessible to all  
  
b.Print();  
d.Print();
```

- `D::Print()` cannot access `B::data_` as it is private

protected Access

```
class B {  
protected: // Accessible to child  
           // Inaccessible to others  
    int data_;  
public: // ...  
    void Print() { cout << "B Object: ";  
        cout << data_ << endl;  
    }  
};  
class D: public B { int info_; public: // ...  
    void Print() { cout << "D Object: ";  
        cout << data_ << ", "; // Accessible  
        cout << info_ << endl;  
    }  
};  
B b(0);  
D d(1, 2);  
  
b.data_ = 5; // Inaccessible to others  
  
b.Print();  
d.Print();
```

- `D::Print()` can access `B::data_` as it is protected



Why do we need protected access?

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

- **Handling Encapsulation:** Encapsulation, the first principle of OOAD, can be enforced in a single class by **private** and **public** access specifiers
 - **private** hides the state (data) of the object and **public** allows the service (method / interface) to be exposed
 - We fine-grain this by **get/set** paradigm to achieve effective information hiding
 - Further **friend** provides a way to sneak through encapsulation for *easy yet safe coding*
- **Encapsulation-Inheritance Conflict:** The above approach to Encapsulation conflicts with Inheritance, the second principle of OOAD

What should be the access specification for data members of a Base class?

 - If they are **public**, the encapsulation is lost for the base class objects
 - If they are **private**, even the derived class methods cannot access them
 - So the derived class object contains the base class data members but cannot access them

Notably, the state of the derived class object depends on the state of its base class part
 - The **get/set** paradigm does not work as it is clumsy and creates an encapsulation hole like **public** if used for all data members
- **Solution:** The **protected** access specifier provides a neat solution by making **protected** base class members available to the derived class while being hidden from the rest of the world
- **Caveat:** **protected** specifier still does not solve all situations and we need to use **friend** to provide a way to sneak through encapsulation as the next example illustrates



Streaming

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

Streaming in B

```

class B { protected: int data_;
public:
    friend ostream& operator<<(ostream& os,
        const B& b) { os << "B Object: ";
        os << b.data_ << endl;
        return os;
    }
};

class D: public B { int info_;
public:
    //friend ostream& operator<<(ostream& os,
    //    const D& d) { os << "D Object: ";
    //    os << d.data_ << ' ' << d.info_ << endl;
    //    return os;
    //}
};

B b(0);      cout << b; // Printed a B object
D d(1, 2);   cout << d; // Printed a B object

```

B Object: 0

B Object: 1

- d printed as a B object; info_ missing

Streaming in B & D

```

class B { protected: int data_;
public:
    friend ostream& operator<<(ostream& os,
        const B& b) { os << "B Object: ";
        os << b.data_ << endl;
        return os;
    }
};

class D: public B { int info_;
public:
    friend ostream& operator<<(ostream& os,
        const D& d) { os << "D Object: ";
        os << d.data_ << ' ' << d.info_ << endl;
        return os;
    }
};

B b(0);      cout << b; // Printed a B object
D d(1, 2);   cout << d; // Printed a D object

```

B Object: 0

D Object: 1 2

- d printed as a D object as expected



Constructor and Destructor

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

**Constructor &
Destructor**

Object Lifetime

Module Summary

Constructor and Destructor



Constructor and Destructor

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access

Streaming

Constructor & Destructor

Object Lifetime

Module Summary

- **Derived ISA Base**
- **Constructor-Destructor**
 - **Derived** class *does not inherit* the **Constructors** and **Destructor** of **Base** class but *must have access to them*
 - **Derived** class *must provide* its own **Constructors** and **Destructor**
 - **Derived** class *cannot override* or *overload* a **Constructor** or the **Destructor** of **Base** class
- **Construction-Destruction**
 - A *constructor* of the **Derived** class *must first* call a *constructor* of the **Base** class to construct the **Base** class instance of the **Derived** class
 - The *destructor* of the **Derived** class *must* call the *destructor* of the **Base** class to destruct the **Base** class instance of the **Derived** class



Constructor and Destructor

Module M23

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

protected Access Streaming

Constructor & Destructor

Object Lifetime

Module Summary

```
class B { protected: int data_; public:
    B(int d = 0) : data_(d) { cout << "B::B(int): " << data_ << endl; }
    ~B() { cout << "B::~~B(): " << data_ << endl; }
    // ...
};

class D: public B { int info_; public:
    D(int d, int i) : B(d), info_(i) // ctor-1: Explicit construction of Base
    { cout << "D::D(int, int): " << data_ << ", " << info_ << endl; }
    D(int i) : info_(i) // ctor-2: Default construction of Base
    { cout << "D::D(int): " << data_ << ", " << info_ << endl; }
    ~D() { cout << "D::~~D(): " << data_ << ", " << info_ << endl; }
    // ...
};

B b(5);
D d1(1, 2); // ctor-1: Explicit construction of Base
D d2(3); // ctor-2: Default construction of Base
```

Object Layout

Object b

5

Object d1



Object d2





Object Lifetime

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

NPTEL

Object Lifetime



Object Lifetime

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access
Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

```

class B { protected: int data_; public:
    B(int d = 0) : data_(d) { cout << "B::B(int): " << data_ << endl; }
    ~B() { cout << "B::~~B(): " << data_ << endl; }
    // ...
};

class D: public B { int info_; public:
    D(int d, int i) : B(d), info_(i) // ctor-1: Explicit construction of Base
    { cout << "D::D(int, int): " << data_ << ", " << info_ << endl; }
    D(int i) : info_(i) // ctor-2: Default construction of Base
    { cout << "D::D(int): " << data_ << ", " << info_ << endl; }
    ~D() { cout << "D::~~D(): " << data_ << ", " << info_ << endl; }
    // ...
};

B b;
D d1(1, 2); // ctor-1: Explicit construction of Base
D d2(3); // ctor-2: Default construction of Base

```

Construction O/P

```

B::B(int): 0 // Object b
B::B(int): 1 // Object d1
D::D(int, int): 1, 2 // Object d1
B::B(int): 0 // Object d2
D::D(int): 0, 3 // Object d2

```

Destruction O/P

```

D::~~D(): 0, 3 // Object d2
B::~~B(): 0 // Object d2
D::~~D(): 1, 2 // Object d1
B::~~B(): 1 // Object d1
B::~~B(): 0 // Object b

```

- First construct base class object, then derived class object
- First destruct derived class object, then base class object



Module Summary

Module M23

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

protected
Access

Streaming

Constructor &
Destructor

Object Lifetime

Module Summary

- Understood the need and use of **protected** Access specifier
- Discussed the Construction and Destruction process of class hierarchy and related Object Lifetime

NPTEL



Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

Programming in Modern C++

Module M24: Inheritance: Part 4: Phone Hierarchy

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M24

Partha Pratim
Das

Objectives & Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

- Understood the need and use of **protected** access specifier
- Discussed the Construction and Destruction process of class hierarchy and related Object Lifetime



Module Objectives

Module M24

Partha Pratim
Das

Objectives & Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

- Model a hierarchy of phones using inheritance

NPTEL



Module Outline

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

- 1 Objectives & Outlines
- 2 ISA Hierarchy Design by Inheritance
- 3 Helper Classes
- 4 Hierarchy of Phones by Interfaces
- 5 Interfaces & State Variables of Phones
 - Landline Phone
 - Mobile Phone
 - Smart Phone
- 6 Refactoring
- 7 Hierarchy Integration
 - Extended Hierarchy of Phones
- 8 Module Summary



ISA Hierarchy Design by Inheritance

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

ISA Hierarchy Design by Inheritance



Approach to Modeling Hierarchy

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

- Identify the **Concepts and their ISA relationships** to define the hierarchy: model with public inheritance
- Identify and model **Helper classes** - lower level UDTs to define components
- Identify the **Interface** of each concept: signatures of public member functions
- Identify the **State Variables** of each concept: types of private / protected data members (also member functions used for ease of implementation)
- **Refactor** common data members and member functions between specialized and generalized classes to link the classes by inheritance
- **Integrate the hierarchy** with abstract (pure) interface
- Explore **extendability**
- *We illustrate with the phone hierarchy*



Helper Classes

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

Helper Classes



Helper Classes

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

Class	Description
<code>class PhoneNumber</code>	12-digit phone number
<code>class Name</code>	Subscriber Name (as string)
<code>class Photo</code>	Image & Subscriber Name as alt text
<code>class RingTone</code>	Audio & ring tone name
<code>class Contact</code>	<code>PhoneNumber</code> , <code>Name</code> , and <code>Photo</code> (optional) of a contact
<code>class AddressBook</code>	List of contacts



Hierarchy of Phones by Interfaces

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

NPTEL

Hierarchy of Phones by Interfaces



Hierarchy of Phones

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

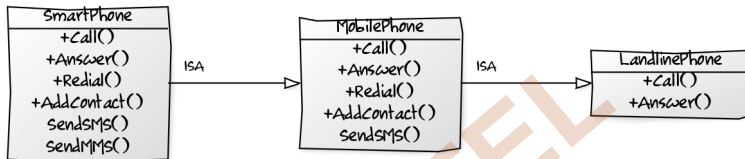
Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary



- **MobilePhone ISA LandlinePhone**
 - LandlinePhone is *generalization*
 - MobilePhone is *specialization*
 - MobilePhone inherits the properties of LandlinePhone
- **SmartPhone ISA MobilePhone**
 - MobilePhone is *generalization*
 - SmartPhone is *specialization*
 - SmartPhone inherits the properties of MobilePhone
- **ISA is *transitive***



Interfaces of Phones

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

- **Landline Phone**

- Call: By dial / keyboard
- Answer
- Caller ID (with special attached device)

- **Mobile Phone**

- Call: By keyboard – shows number
 - ▷ By Number
 - ▷ By Name
- Answer
- Caller ID
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name

- **Smart Phone**

- Call: By touchscreen – shows number & photo
 - ▷ By Number
 - ▷ By Name
- Answer
- Caller ID
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name
 - ▷ Photo

- There exists a substantial overlap between the functionalities of the phones
- A mobile phone is more capable than a land line phone and can perform (almost) all its functions
- A smart phone is more capable than a mobile phone and can perform (almost) all its functions
- These phones belong to a **Specialization / Generalization Hierarchy**



Interfaces & State Variables of Phones

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

**Interfaces &
State Variables of
Phones**

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

NPTEL

Interfaces & State Variables of Phones



Interface & State Variable: Landline Phone

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy
Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

- **Landline Phone**

- Call: By dial / keyboard
- Answer

```
class LandlinePhone {  
    PhoneNumber number_;  
    Name subscriber_;  
    RingTone rTone_;  
  
public:  
    LandlinePhone(const char *num, const char *subs);  
  
    void Call(const PhoneNumber *p);  
  
    void Answer();  
  
    friend ostream& operator<<(ostream& os, const LandlinePhone& p);  
};
```



Interface & State Variable: Mobile Phone

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

● Mobile Phone

- Call: By keyboard – shows number
 - ▷ By Number
 - ▷ By Name
- Answer
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name

```
class MobilePhone {
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();

public:
    MobilePhone(const char *num, const char *subs);
    void Call(PhoneNumber *p);
    void Call(const Name& n);
    void Answer();
    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
                    const char *subs = 0);

    friend ostream& operator<<(ostream& os, const MobilePhone& p);
};
```



Interface & State Variable: Smartphone

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

● Smart Phone

- Call: By touchscreen – shows number & photo
 - ▷ By Number
 - ▷ By Name
- Answer
- Redial
- Set Ring Tone
- Add Contact
 - ▷ Number
 - ▷ Name
 - ▷ Photo

```
class SmartPhone {  
    PhoneNumber number_;  
    Name subscriber_;  
    RingTone rTone_;  
    AddressBook aBook_;  
    PhoneNumber *lastDial_;  
    void SetLastDialed(const PhoneNumber& p);  
    void ShowNumber();  
    unsigned int size_;  
    void DisplayPhoto();  
  
public:  
    SmartPhone(const char *num, const char *subs);  
  
    void Call(PhoneNumber *p);  
    void Call(const Name& n);  
  
    void Answer();  
  
    void ReDial();  
    void SetRingTone(RingTone::RINGTONE r);  
    void AddContact(const char *num = 0,  
                    const char *subs = 0);  
  
    friend ostream& operator<<(ostream& os, const MobilePhone& p);  
};
```



Refactoring

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

NPTEL

Refactoring



MobilePhone ISA LandlinePhone

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

```
class LandlinePhone { protected:
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;

public:
    LandlinePhone(const char *num,
                  const char *subs) :
        number_(num), subscriber_(subs),
        rTone_() { }
    void Call(const PhoneNumber *p);

    void Answer();

    friend ostream& operator<<(ostream& os,
                              const LandlinePhone& p);
};
```

```
class MobilePhone : public LandlinePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();

public:
    MobilePhone(const char *num,
                const char *subs) :
        LandlinePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    //void Answer();                 // Inherited
    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
                    const char *subs = 0);
    friend ostream& operator<< (ostream& os,
                               const MobilePhone& p);
};
```



MobilePhone ISA LandlinePhone

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy
Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

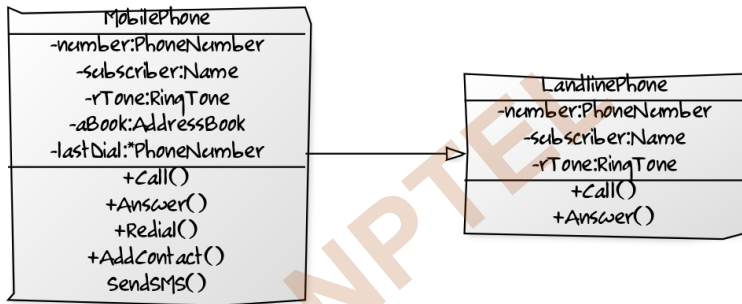
Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary





SmartPhone ISA MobilePhone

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone
Mobile Phone
Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

```
class MobilePhone : public LandlinePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();

public:
    MobilePhone(const char *num,
                const char *subs) :
        LandlinePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    //void Answer();                 // Inherited
    void ReDial();
    void SetRingTone(RingTone::RINGTONE r);
    void AddContact(const char *num = 0,
                    const char *subs = 0);
    friend ostream& operator<< (ostream& os,
                               const MobilePhone& p);
};

class SmartPhone : public MobilePhone { protected:
    //PhoneNumber number_;
    //Name subscriber_;
    //RingTone rTone_;
    AddressBook aBook_;
    //PhoneNumber *lastDial_;
    //void SetLastDialed(const PhoneNumber& p);
    //void ShowNumber();
    unsigned int size_;
    void DisplayPhoto()

public:
    SmartPhone(const char *num,
               const char *subs) :
        MobilePhone(num, subs), // Base ctor
        lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Override
    //void Answer();
    void ReDial();                   // Override
    //void SetRingTone(RingTone::RINGTONE r);
    //void AddContact(const char *num = 0,
                    //const char *subs = 0);
    friend ostream& operator<< (ostream& os,
                               const SmartPhone& p);
};
```




Hierarchy Integration

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

**Hierarchy
Integration**

Extended Hierarchy
of Phones

Module Summary

Hierarchy Integration



Hierarchy Integration

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary

```
// Abstract Base Class - A Pure Interface
```

```
class Phone { public:
    virtual void Call(const PhoneNumber *p) = 0;
    virtual void Answer() = 0;
    virtual void ReDial() = 0;
};

class LandlinePhone: public Phone {
protected:
    PhoneNumber number_;
    Name subscriber_;
    RingTone rTone_;
public:
    LandlinePhone(const char *num,
        const char *subs) :
        number_(num), subscriber_(subs),
        rTone_() { }
    // Implementations for interfaces
    void Call(const PhoneNumber *p);
    void Answer();
    // Dummy implementation not for use
    void ReDial()
    { cout << "Not implemented" << endl; }
    friend ostream& operator<<(ostream& os,
        const LandlinePhone& p);
};
```

Programming in Modern C++

```
class MobilePhone : public LandlinePhone { protected:
    AddressBook aBook_;
    PhoneNumber *lastDial_;
    void SetLastDialed(const PhoneNumber& p);
    void ShowNumber();
public:
    MobilePhone(const char *num, const char *subs) :
        LandlinePhone(num, subs), lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Overload
    void ReDial();                   // Override
    friend ostream& operator<< (ostream& os,
        const MobilePhone& p);
};

class SmartPhone : public MobilePhone {
protected: unsigned int size_;
    void DisplayPhoto();
public:
    SmartPhone(const char *num, const char *subs) :
        MobilePhone(num, subs), lastDial_(0) { }
    void Call(const PhoneNumber *p); // Override
    void Call(const Name& n);        // Override
    void ReDial();                   // Override
    friend ostream& operator<< (ostream& os,
        const SmartPhone& p);
};
```

Partha Pratim Das

M24.21



Extended Hierarchy of Phones

Module M24

Partha Pratim Das

Objectives & Outlines

ISA Hierarchy Design by Inheritance

Helper Classes

Hierarchy of Phones by Interfaces

Interfaces & State Variables of Phones

Landline Phone

Mobile Phone

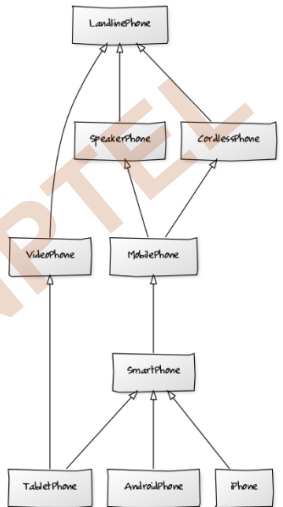
Smart Phone

Refactoring

Hierarchy Integration

Extended Hierarchy of Phones

Module Summary





Module Summary

Module M24

Partha Pratim
Das

Objectives &
Outlines

ISA Hierarchy
Design by
Inheritance

Helper Classes

Hierarchy of
Phones by
Interfaces

Interfaces &
State Variables of
Phones

Landline Phone

Mobile Phone

Smart Phone

Refactoring

Hierarchy
Integration

Extended Hierarchy
of Phones

Module Summary

- Using the Phone Hierarchy as an example analyzed the design process with inheritance

NPTEL



Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

Programming in Modern C++

Module M25: Inheritance: Part 5: `private` & `protected` Inheritance

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Module Recap

Module M25

Partha Pratim
Das

Objectives & Outlines

Inheritance in
C++

private
Inheritance

Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- Using the Phone Hierarchy as an example analyzed the design process with inheritance

NPTEL



Module Objectives

Module M25

Partha Pratim
Das

Objectives & Outlines

Inheritance in
C++

private
Inheritance

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- Explore restricted forms of inheritance (`private` and `protected`) in C++ and their semantic implications

NPTEL



Module Outline

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- 1 Objectives & Outlines
- 2 Inheritance in C++
- 3 private Inheritance
 - Uncopyable
 - HAS-A
- 4 protected Inheritance
- 5 Visibility
- 6 Examples
- 7 Module Summary



Inheritance in C++

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

Inheritance in C++



Inheritance in C++: Semantics

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

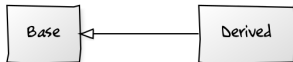
protected Inheritance

Visibility

Examples

Module Summary

- Derived ISA Base



```
class Base; // Base Class = Base
class Derived: public Base; // Derived Class = Derived
```

- Use keyword `public` after class name to denote inheritance
- Name of the Base class follow the keyword



Inheritance Exercise: What is the output?

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

protected Inheritance

Visibility

Examples

Module Summary

```
class B {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};

class C {
public:
    C() { cout << "C "; }
    ~C() { cout << "~C "; }
};

class D : public B {
    C data_; // Embedded Object
public:
    D() { cout << "D " << endl; } // Intrinsic Base Class Object
    ~D() { cout << "~D "; }
};

int main() {
    D d;
}
```



Inheritance Exercise: What is the output?

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

protected Inheritance

Visibility

Examples

Module Summary

```
class B {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};
class C {
public:
    C() { cout << "C "; }
    ~C() { cout << "~C "; }
};
class D : public B {
    C data_; // Embedded Object
public:
    D() { cout << "D " << endl; } // Intrinsic Base Class Object
    ~D() { cout << "~D "; }
};
int main() {
    D d;
}
```

Output:

```
B C D          // First base class object, then embedded object, finally self
~D ~C ~B
```



private Inheritance

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

**private
Inheritance**

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

private Inheritance



private Inheritance

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- **private** Inheritance

- Definition

```
class Base;
```

```
class Derived: private Base;
```

- Use keyword **private** after class name

- Name of the Base class follow the keyword

- **private** inheritance **does not** mean generalization / specialization



private Inheritance

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

public Inheritance

```
class Person { ... };

class Student:
    public Person { ... };

void eat(const Person& p);    // anyone can eat
void study(const Student& s); // only students study

Person p; // p is a Person
Student s; // s is a Student

eat(p);    // fine, p is a Person
eat(s);    // fine, s is a Student,
           // and a Student is-a Person

study(s); // fine
study(p); // error! p isn't a Student
```

- Compilers **converts** a derived class object (**Student**) into a base class object (**Person**) if the inheritance relationship is **public**

private Inheritance

```
class Person { ... };

class Student: // inheritance is now private
    private Person { ... };

void eat(const Person& p);    // anyone can eat
void study(const Student& s); // only students study

Person p; // p is a Person
Student s; // s is a Student

eat(p);    // fine, p is a Person
eat(s);    // error! a Student isn't a Person
```

- Compilers **will not convert** a derived class object (**Student**) into a base class object (**Person**) if the inheritance relationship is **private**



Uncopyable Class

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance
Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- Suppose we want to design a `MyClass` every object must be *unique*. That is instance objects must not be copied (the class needs to be *Uncopyable*)

```
MyClass m1;  
MyClass m2;
```

```
MyClass m3(m1); // attempt to copy m1 - should not compile!  
m1 = m2;        // attempt to copy m2 - should not compile!
```

- Naturally, we do not want to provide copy constructor or copy assignments operator. But that does not work as the compiler will provide free versions of these functions. How to stop that?

```
class MyClass {  
public:  
    ...  
private:  
    ...  
    MyClass(const MyClass&); // declarations only  
    MyClass& operator=(const MyClass&);  
};
```

The last trick is not to provide the implementations (bodies) of these copy functions.

- With the above any global function or other class would not be able to copy - there will be *compilation error*; and any member function of `MyClass` will get *linker error* on missing implementation
- This is, of course, not an elegant solution and has to depend on the programmer to do things right for every such *Uncopyable* class. *private* inheritance helps out



Uncopyable

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance
Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- `class Uncopyable` is designed as a root class that can make copy functions of any child class `private`

```
class Uncopyable { protected: // allow construction and destruction of derived objects ...
    Uncopyable() { }
    ~Uncopyable() { }
private:
    Uncopyable(const Uncopyable&); // ... but prevent copying
    Uncopyable& operator=(const Uncopyable&);
};
```
- Any class that inherits from `class Uncopyable`, will not have copy functionality:

```
class MyClass: private Uncopyable { // class no longer declares copy ctor or copy assign. operator
    // ...
    void ProhibitiveCopy() { MyClass test1, test2; // Member functions cannot perform copy
        MyClass test3(test1); // Error 1: 'Uncopyable::Uncopyable' : cannot access private member
        test2 = test1; // Error 2: 'Uncopyable::operator =' : cannot access private member
    }
};
```

```
int main() { MyClass test1, test2; // Global functions cannot perform copy
    MyClass test3(test1); // Error 1: 'Uncopyable::Uncopyable' : cannot access private member
    test2 = test1; // Error 2: 'Uncopyable::operator =' : cannot access private member
}
```
- The inheritance from `Uncopyable` need not be `public` (though it will work), hence using `private` we express that it is purely for implementation - not for modeling **ISA** that actually does not exist
- You may also use the `boost::noncopyable`
- **C++11** provides an explicit support by `delete` to stop compilers from providing free functions



Car HAS-A Engine: Composition OR private Inheritance?

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

protected Inheritance

Visibility

Examples

Module Summary

Simple Composition

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};

class Car {
public:
    // Initializes this Car with 8 cylinders
    Car() : e_(8) { }

    // Start this Car by starting its Engine
    void start() { e_.start(); }
private:
    Engine e_; // Car has-a Engine
};

int main() {
    Car c;

    c.start();
}
```

private Inheritance

```
#include <iostream>
using namespace std;

class Engine {
public:
    Engine(int numCylinders) { }
    void start() { } // Starts this Engine
};

class Car : private Engine { // Car has-a Engine
public:
    // Initializes this Car with 8 cylinders
    Car() : Engine(8) { }

    // Start this Car by starting its Engine
    using Engine::start;
};

int main() {
    Car c;

    c.start();
}
```



private Inheritance

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

- For **HAS-A**: Use composition when you can, `private` inheritance when you have to

- **Private inheritance means nothing during software design, only during software implementation**

- **Private inheritance means `is-implemented-in-terms` of.** It is usually inferior to composition, but it makes sense when a derived class needs access to protected base class members or needs to redefine inherited virtual functions

– Scott Meyers in Item 32, Effective C++ (3rd. Edition)



protected Inheritance

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

`private`
Inheritance

Uncopyable
HAS-A

`protected`
Inheritance

Visibility

Examples

Module Summary

protected Inheritance



protected Inheritance

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

protected Inheritance

Visibility

Examples

Module Summary

- **protected** Inheritance

- Definition

- ```
class Base;
```

- ```
class Derived: protected Base;
```

- Use keyword **protected** after class name

- Name of the Base class follow the keyword

- **protected** inheritance **does not** mean generalization / specialization

● **Private inheritance means something entirely different (from public inheritance), and protected inheritance is something whose meaning eludes me to this day**

– *Scott Meyers in Item 32, Effective C++ (3rd. Edition)*



Visibility

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

`private`
Inheritance

Uncopyable
HAS-A

`protected`
Inheritance

Visibility

Examples

Module Summary

Visibility

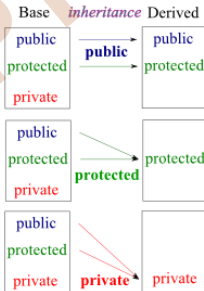
NPTEL



Visibility across Access and Inheritance

- Visibility Matrix

		Inheritance		
		public	protected	private
Visibility	public	public	protected	private
	protected	protected	protected	private
	private	private	private	private





Use and Examples

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

`private`
Inheritance

Uncopyable
HAS-A

`protected`
Inheritance

Visibility

Examples

Module Summary

Use and Examples



Inheritance Exercise: What is the output?

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable

HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

```
class B {  
protected:  
    B() { cout << "B "; }  
    ~B() { cout << "~B "; }  
};  
class C : public B {  
protected:  
    C() { cout << "C "; }  
    ~C() { cout << "~C "; }  
};  
class D : private C {  
    C data_;  
public:  
    D() { cout << "D " << endl; }  
    ~D() { cout << "~D "; }  
};  
  
int main() {  
    D d;  
}
```



Inheritance Exercise: What is the output?

Module M25

Partha Pratim
Das

Objectives &
Outlines

Inheritance in
C++

private
Inheritance

Uncopyable
HAS-A

protected
Inheritance

Visibility

Examples

Module Summary

```
class B {  
protected:  
    B() { cout << "B "; }  
    ~B() { cout << "~B "; }  
};  
class C : public B {  
protected:  
    C() { cout << "C "; }  
    ~C() { cout << "~C "; }  
};  
class D : private C {  
    C data_;  
public:  
    D() { cout << "D " << endl; }  
    ~D() { cout << "~D "; }  
};  
  
int main() {  
    D d;  
}
```

Output:

```
B C B C D  
~D ~C ~B ~C ~B
```



Inheritance Exercise: Access Rights

Module M25

Partha Pratim Das

Objectives & Outlines

Inheritance in C++

private Inheritance

Uncopyable

HAS-A

protected Inheritance

Visibility

Examples

Module Summary

Inaccessible Members

```
class A { private: int x;
         protected: int y;
         public: int z;
};
class B : public A { private: int u;
                    protected: int v;
                    public: int w; void f() { x; }
};
class C: protected A { private: int u;
                      protected: int v;
                      public: int w; void f() { x; }
};
class D: private A { private: int u;
                   protected: int v;
                   public: int w; void f() { x; }
};
class E : public B { public: void f() { x; u; }
};
class F : public C { public: void f() { x; u; }
};
class G : public D { public: void f() { x; y; z; u; }
};
```

Accessible Members

```
void f(A& a,
      B& b, C& c, D& d,
      E& e, F& f, G& g) {
    a.z;
    b.z;
    b.w;
    c.w;
    d.w;
    e.z;
    e.w;
    f.w;
    g.w;
}
```



Module Summary

Module M25

Partha Pratim Das

Objectives &
Outlines

Inheritance in
C++

`private`
Inheritance

Uncopyable
HAS-A

`protected`
Inheritance

Visibility

Examples

Module Summary

- Introduced restricted forms of inheritance and protected specifier
- Discussed how `private` inheritance is used for *Implemented-As* Semantics



Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

Programming in Modern C++

Tutorial T05: Mixing C and C++ Code: Part 1: Issues and Resolutions

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur

ppd@cse.iitkgp.ac.in

All url's in this module have been accessed in September, 2021 and found to be functional



Tutorial Objectives

Tutorial T05

Partha Pratim
Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler

Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- Due to legacy, reuse and several business compulsions, we often need to mix C and C++ codes in the same project
- So we need to learn how to write programs mixing C and C++?



Tutorial Outline

Tutorial T05

Partha Pratim
Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

1 Mixing C and C++ Codes

- Why Mix C/C++?
- Build all in C++
- Mix C and C++
 - Static Initialization
 - Compiler Compatibility
 - Linkage Issues
 - Exception Issues

2 Common Code Mix Scenarios

- How do I call a C function from C++?
- How do I call a C++ function from C?
- How do I use Pointers to C / C++ Functions?
- How do I include a C Header File?
 - System Header File
 - Non-System Header File

3 Tutorial Summary



Mixing C and C++ Codes

Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

Mixing C and C++ Codes

Source: Accessed 16-Sep-21

[How to mix C and C++, ISO CPP](#)

[Mixing C and C++ Code in the Same Program, Oracle](#)

[C++ Core Guidelines: Mixing C with C++](#)

[Mixing Code in C, C++, and FORTRAN on Unix](#)

Programming in Modern C++

Partha Pratim Das

T05.4



Why do we need to mix C and C++ Codes?

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler

Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- Primary reason is **legacy and reuse**. There are possibly trillion lines of well-tested C code available. So reusing them in and / or with C++ needs mixing of codes
- Mixing of codes is actually often needed not only across C and C++, it may be used across C and Python, C# and Python, C++ and Java, and so on to get the **best of both languages** (C/C++ is efficient, C is lightweight for embedded programming, Python has rich libraries and good for web, Java is good for applications with GUI etc.) and be able **to use the available proven libraries**. Actually, we may mix *more than two languages*
- Here are some informative articles on projects using multiple languages:
 - [Polyglot programming – development in multiple languages](#), Computer World, 2009
 - [How do you use different coding languages in one program?](#), Quora, 2015 and several other in Quora
 - [A Large Scale Study of Multiple Programming Languages and Code Quality](#), IEEE, 2016
 - [On multi-language software development, cross-language links and accompanying tools: a survey of professional software developers](#), Springer, 2017
- **We restrict the discussions here on mixing C and C++ only**



Why do we need to mix C and C++ Codes?

- **There are two typical situations:**

- *Both C header and C source files are available and editable*: An existing project that needs to be migrated to C++ in full or parts (needs to be reused). Two options:
 - ▷ **Mix C and C++ codes**: Compile C code with C compiler, and C++ code with C++ compiler and then link by C++ (**how?**)
 - ▷ **Build all in C++**: Build both C and C++ codes with C++ compiler and link
- *Only C header files are available*: For third party library where source is already pre-compiled. In fact, *the header file too may not be editable*. For example,
 - ▷ To write a C++ program/library that does scientific calculations, we would possibly use GSL (**GNU Scientific Library**), and it is written in C
 - ▷ The C game codes called from the C++ graphics engine

We would like to wrap all the C functions to use in nice C++ style functions, perhaps with the necessary exceptions and returning a `std::string` instead of having to pass a `char*` buffer as argument. Now we have only one option:

- ▷ **Mix C and C++ codes**: Compile C code with C compiler, and C++ code with C++ compiler and then link by C++ (**how?**)



Build C and C++ Codes as C++

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Build as C++:** In a mixed code project where all header and source files are *available and editable*, we can *compile all the code* (even the C-style code) using a *C++ compiler*. For example, using *g++* for both *.c* and *.cpp* files. That eliminates the need to mix C and C++
- However, it is not easy to build the C code by C++ compiler unless the C code strictly uses the common subset of C and C++ (Check the Tutorial on *Compatibility of C and C++* for details). For example, consider the simple C program below where difference of behavior in C and C++ compilers are marked in different colors:

```
/* cStyle.c */
#include <stdio.h>
int main() {
    double sq2=sqrt(2); // (1): math.h missing. Warning in C89. Error in C++98
    printf("sizeof('a'): %d",
           sizeof('a')); // (2): 'a' is int in C, char in C++. Outputs 4 in C89. Outputs 1 in C++98
    char c;
    void* pv = &c;
    int* pi = pv; // (3): Implicit conversion from void* to int*. Okay in C89. Error in C++98
    int class = 5; // (4): class is a reserved word in C++. Okay in C89. Error in C++98
}
```

- So the **C code needs to be ported for C++**



Build C and C++ Codes as C++

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Build as C++:** While building a C/C++ project (both C and C++ codes) with C++ is preferable from language perspective it has a number of shortcomings from engineering viewpoint
 - The *C-style code may need porting* as the *C++ compiler is more strict* - as we have seen in the example
 - *Porting may involve substantial cost* in terms of developer effort as well as project time. This may *not be affordable from the business perspectives*
 - With porting we also need to *create new testplan for C++*, perform *extensive testing* to match the regression. This involves further cost in terms of tester effort as well as project time. This may *not be affordable from the business perspectives*
 - If we are porting a stable C code, even after regression clean and testing, it is *likely to break some of the existing functionality* in the software or even in customer's code if the C code is part of a library provided to the customer. This too may *not be affordable from the business perspectives*
 - So building as C++ is *feasible only in some select situations* though it is *preferred*



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- When we mix codes, that is, compile C code by C Compiler and C++ code by C++ Compiler - all into `.o` files - we expect to be free from language-specific issues in *individual translation units*
- However, issues arise as we work with different versions of compilers, link the `.o` files of translation units, and as control flows across units during execution, and so on:
 - *Static Initialization Issues*: While compiling `main()`, static initialization is handled differently in C and C++
 - *Compiler Incompatibility Issues*: The compilers may have *incompatibility* in *calling conventions*, *definitions of basic types* (like `int`, pointer), *runtime library*, etc.
 - *C Library Incompatibility Issues*: If the C++ compiler provides its own versions of the C headers, the headers used by the C compiler must be compatible
 - *Linkage Issues*: C and C++ linkage conventions differ
 - *Exception Issues*: C and C++ use drastically different exception models
 - *Scope of struct Issue*: Scoping of *nested struct* differ between C and C++.Check the Tutorial on *Compatibility of C and C++* for details



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Static Initialization Issues:** In C and C++ both the `static` variables are constructed and initialized before `main()`. But they have different semantics and handling for `static`

- In C, a `static` initializer must be a constant
- In C++, a `static` variable must be constructed – its constructor must get called

So the following code compiles in C++, but fails in C

```
#include <stdio.h>
int init(void) { return 10; }

static int i = init(); /* Error in C: initializer element is not constant. Okay in C++ */

int main() { printf("i = %d", i); }
```

Hence,

- C++ compiler generates an additional `Start` function, where all *global function calls* (including constructors) are executed before `main` starts
- C compiler does not generate such `Start` function, `main` starts as soon as it is loaded

So,

- **RULE 1:** Use **C++ compiler** when **compiling `main()`**



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Compiler and C Library Incompatibility Issues:** To alleviate the problems outlined, we should

- Use compilers (preferably) from the same vendor (say, `gcc`)
- Have **same / compatible versions** (for example, use the same calling conventions, define basic types such as `int`, `float` or pointer in the same way)
- **C runtime library** must also be **compatible with the C++ compiler**
- If the C++ compiler provides its own versions of the C headers, the versions of those headers used by the C compiler must be compatible

So,

- **RULE 2: C and C++ compilers must be compatible**



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Linkage Issues:** C and C++ linkage conventions differ

- In C

- ▷ Every function is in global scope
- ▷ Every function name is unique

Hence the **C function name** is used by the linker to identify the function across units

- In C++

- ▷ A function may be in multiple scopes
 - Global Scope
 - Non-Static Member Function in class Scope
 - Static Member Function in class Scope
 - namespace Scope

- ▷ A function may be overloaded in any of these scopes

Hence the C++ function name is not unique. The linker, therefore, uses the combination of the **C++ function name** and **signature (parameter types)** to create unique identity, called **mangled name**



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Linkage Issues:** Consider C++ code with 2 overloads of `print()` in multiple scopes

```
#include <iostream>
using namespace std;
void print(int) { cout << "int" << endl; } // Global
void print(double) { cout << "double" << endl; }
class MyClass { int i; double d; const char *pc; public:
    MyClass(int _i = 1, double _d = 1.1, const char *_pc = "Hello") : i(_i), d(_d), pc(_pc) { }
    void print(int) { cout << "MyClass int " << i << endl; } // Class member
    void print(double) { cout << "MyClass double " << d << endl; }
};
class MyOtherClass { public:
    static void print(int) { cout << "MyOtherClass int " << endl; } // Class static member
    static void print(double) { cout << "MyOtherClass double " << endl; }
};
namespace MySpace {
    void print(int) { cout << "MySpace int" << endl; } // namespace member
    void print(double) { cout << "MySpace double" << endl; }
}
int main() { MyClass a;
    print(10); print(10.10);
    a.print(10); a.print(10.10);
    MyOtherClass::print(10); MyOtherClass::print(10.10);
    MySpace::print(10); MySpace::print(10.10);
}
```



Mixing C and C++ Codes: Issues and Remedies

- **Linkage Issues:** The mangled and un-mangled names of functions are:

Function	gcc 6.3.0	msvc 18.00	Mangled?
// Global: Overloaded			
print(int)	--Z5printi	?print@@YAXH@Z	Yes
print(double)	--Z5printd	?print@@YAXN@Z	Yes
// Class member: Overloaded			
MyClass::print(int)	--ZN7MyClass5printEi	?print@MyClass@@QAEXH@Z	Yes
MyClass::print(double)	--ZN7MyClass5printEd	?print@MyClass@@QAEXN@Z	Yes
// Class static member: Overloaded			
MyOtherClass::print(int)	--ZN12MyOtherClass5printEi	?print@MyOtherClass@@SAXH@Z	Yes
MyOtherClass::print(double)	--ZN12MyOtherClass5printEd	?print@MyOtherClass@@SAXN@Z	Yes
// namespace member: Overloaded			
MySpace::print(int)	--ZN7MySpace5printEi	?print@MySpace@@YAXH@Z	Yes
MySpace::print(double)	--ZN7MySpace5printEd	?print@MySpace@@YAXN@Z	Yes
// Global: Not Overloaded			
main()	_main	_main	No

Therefore C and C++ compilers need to handle the names differently. *As C compiler does not know about mangling, we need to tell the C++ compiler not to mangle the names in the C context*



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Linkage Issues:** The `extern "C"` linkage specifier can prevent the C++ compiler from mangling the names. Declaring a function within `extern "C"` in the code, we can call a C function from C++, or a C++ function from C. We may use `extern "C"`

- for each function

```
extern "C" void foo(int);
```

- for each function in a scope

```
extern "C" {  
    void foo(int);  
    double bar(double);  
};
```

- or for the entire header file by using include guards

```
#ifndef __cplusplus  
extern "C" {  
#endif  
    void foo(int);  
    double bar(double);  
    ...  
#ifdef __cplusplus  
}  
#endif
```

Note that the macro `__cplusplus` is defined when the C++ compiler is used. Hence, `extern "C"` is not exposed to C compiler (it does not recognize it)



Mixing C and C++ Codes: Issues and Remedies

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **Linkage Issues:** As we have seen, using `extern "C"` within `__cplusplus` guard, we can make a header that works appropriately for C and C++ both. For an illustrative example, you may refer to the video: [C Programming — Advance Topic And Mixing C with C++ Code, YouTube](#) (Accessed 19-Sep-21)

We now have the next rule:

- **RULE 3:** **C++ compiler** should direct the **linking process** (so it can get its special libraries)



Mixing C and C++ Codes: Issues and Remedies

[Advanced]

- **Exception Issues:** C and C++ use different exception models

- **Propagating Exceptions**

- ▷ What happens if a *C++ function is called from a C function, and the C++ function throws an exception*? The C++ standard is somewhat vague this
- ▷ In some systems like **Oracle Developer Studio C++** this will work properly. In it *if a C function is active when a C++ exception is thrown, the C function is passed over in the process of handling the exception*

- **Mixing Exceptions with `setjmp` and `longjmp`**

- ▷ The C++ exception mechanism and C++ rules about *destroying objects that go out of scope* are likely to be *violated by a `longjmp`, with unpredictable results*
- ▷ Some C++ experts believe that `longjmp` should not be integrated with exceptions, due to the difficulty of specifying exactly how it should behave.
- ▷ If we must use `longjmp` in C code that we are mixing with C++, we need to *ensure that a `longjmp` does not cross over an active C++ function*



Common Code Mix Scenarios

Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

Common Code Mix Scenarios



Common Code Mix Scenarios

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler

Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- How do I call a C function from C++?
- How do I call a C++ function from C?
 - Non-Member
 - ▷ Global / namespace
 - ▷ static
 - Member
 - ▷ Non-static
 - Non-virtual
 - virtual
 - ▷ static
 - Overloaded

- How do I include a C Header File?
 - System / Standard Library Headers
 - Non-System / User-defined Headers
 - ▷ Editable Headers
 - ▷ Non-Editable Headers
- How do I use Pointers to C / C++ Functions?
- How do I manipulate with objects in a C / C++ mix project?



How do I call a C function from C++?

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler

Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- Declare the C function `extern "C"` (in the C++ code) and call it (from the C or C++ code):

// C++ code

`extern "C" void f(int);` // one way

`extern "C" {` // another way

`int g(double);`

`double h();`

`};`

`void code(int i, double d) {`

`f(i);`

`int ii = g(d);`

`double dd = h();`

`// ...`

`}`

/* C code: */

`void f(int i) {`

`/* ... */`

`}`

`int g(double d) {`

`/* ... */`

`}`

`double h() {`

`/* ... */`

`}`

Note that C code does not need to be edited
So pre-compiled .o file may also be used

- Note that C++ type rules, not C rules, are used. So a function declared `extern "C"` cannot be called with the wrong number of arguments. For example:

// C++ code

`void more_code(int i, double d) {`

`double dd = h(i,d);` // error: unexpected arguments

`// ...`

`}`



How do I call a C++ function from C?: Non-Member and Member Functions

Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- Declare the C+ `extern "C"` (in the C++ code) and call it (from the C or C++ code):

```
// C++ code
extern "C" void f(int);
void f(int i) {
    // ...
}
// This works only for non-member functions
```

```
/* C code: */
void f(int);
void cc(int i) {
    f(i);
    /* ... */
}
```

- To call member functions (including virtual functions) from C, a simple wrapper needs to be provided. For example:

```
// C++ code
class C {
    // ...
    virtual double f(int);
};
// wrapper function
extern "C"
double call_C_f(C* p, int i) {
    return p->f(i);
}
```

```
/* C code: */
double call_C_f(struct C* p, int i);
void ccc(struct C* p, int i) {
    double d = call_C_f(p,i);
    /* ... */
}
```



How do I call a C++ function from C?: Overloaded Functions

Tutorial T05

Partha Pratim
Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- To call overloaded functions from C, wrappers, with distinct names for the C code to use, must be provided. For example:

// C++ code

void f(int);

void f(double);

extern "C" void f_i(int i) { f(i); }

extern "C" void f_d(double d) { f(d); }

/* C code: */

void f_i(int);

void f_d(double);

void cccc(int i, double d) {

 f_i(i);

 f_d(d);

 /* ... */

}

Note that these techniques can be used to call a C++ library from C code even if it is not desirable or possible to modify the C++ headers



How do I use Pointers to C / C++ Functions?

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- A pointer to a function must specify whether it points to a C function or to a C++ function:

```
typedef int (*pfun)(int); // pfun points to a C++ function
extern "C" void foo(pfun); // foo is a C function taking a pointer to a C++ function
extern "C" int g(int)      // g is a C function
...
// Type Mismatch Error!
foo(g);                  // foo called with a pointer to g, a C function
```

- To match the linkage of a pointer-to-function with the functions to which it will point, all declarations in this example needs to be inside `extern "C"` brackets, ensuring that the types match

```
extern "C" {
    typedef int (*pfun)(int);
    void foo(pfun);
    int g(int);
}
...
foo(g); // Types Match. Now OK
```



How can I include a standard C header file in my C++ code?

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler

Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- **#include**ing a standard header file such as `<cstdio>`, is nothing unusual. For example:

// C++ code

```
// Nothing unusual in #include line
#include <cstdio>
int main() {
    // Nothing unusual in the call either
    std::printf("Hello world");
    // ...
}
```

/* C code */

```
/* Compiled using a C++ compiler */
/* Nothing unusual in #include line */
#include <stdio.h>
int main() {
    /* Nothing unusual in the call either */
    printf("Hello world");
    // ...
}
```

- The `std::` part of the `std::printf()` is for the `std namespace` where C++ Standard Library components like `<cstdio>` belongs
- So a C code using the C++ compiler can still use just `printf()` from `<stdio.h>` as C Standard Library belongs to global
- Care is needed if there is difference between the C header and its corresponding C++ version



How can I include a standard C header file in my C++ code?

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- To summarize

	C Header	C++ Header
C Program	Use .h. Example: <code>#include <stdio.h></code> <i>Names in global namespace</i>	Not applicable
C++ Program	Prefix c, no .h. Example: <code>#include <cstdio></code> <i>Names in std namespace</i>	No .h. Example: <code>#include <iostream></code>

- A C std. library header is used in C++ with prefix 'c' and without the .h. These are in std namespace:

```
#include <cmath> // In C it is <math.h>
```

```
...
```

```
std::sqrt(5.0); // Use with std::
```

It is possible that a C++ program include a C header as in C. Like:

```
#include <math.h> // Not in std namespace
```

```
...
```

```
sqrt(5.0); // Use without std::
```

This, however, is not preferred

- **Using .h with C++ header files, like `iostream.h`, is disastrous. These are deprecated. It is dangerous, yet true, that some compilers do not error out on such use. Exercise caution.**



How can I include a non-system C header file in my C++ code?: Editable Case

Tutorial T05

Partha Pratim Das

Objectives & Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- *If the C header is editable*, add the `extern "C" {...}` logic inside the header and guard it by `__cplusplus` to let C compiler to ignore it

```
#ifndef __cplusplus /* C++ compiler notes, C compiler ignores */
extern "C" {
#endif

void f(int i, char c, float x) /* Original Code of the Header */

#ifdef __cplusplus
}
#endif
```

Now the C header may be `#included` without `extern "C"` in the C++ code:

```
// C++ code
// Get declaration for f(int i, char c, float x)
#include "my-C-code.h" // Note: nothing unusual in #include line
int main() {
    f(7, 'x', 3.14); // Note: nothing unusual in the call
    // ...
}
```



How can I include a non-system C header file in my C++ code?: Non-Editable Case

Tutorial T05

Partha Pratim Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- *If the C header is not editable*, the `#include` line must be wrapped in an `extern "C" { /*...*/ }` construct. This tells the C++ compiler that the functions declared in the header file are C functions

```
// C++ code
extern "C" {
    // Get declaration for f(int i, char c, float x)
    #include "my-C-code.h"
}

int main() {
    f(7, 'x', 3.14); // Note: nothing unusual in the call
    // ...
}
```



Tutorial Summary

Tutorial T05

Partha Pratim Das

Objectives &
Outline

Mixing C & C++

Why Mix C/C++?

Build all in C++

Mix C & C++

Static Initialization

Compiler
Compatibility

Linkage Issues

Exception Issues

Common Mix

C from C++

C++ from C

Pointers to Functions

C Header File

System

Non-System

Tutorial Summary

- We have learnt why is it often necessary to mix C and C++ codes in the same project
- We have explored the basic issues of mixing and learnt the ground rules
- In addition to the rules, we have three mechanisms to ease code mixing
 - Use `extern "C"` in C++ for all functions to be called from both C and C++
 - Guard `extern "C"` with `__cplusplus` guard for use with C
 - Provide `wrappers` for C++ data members, member functions, and overloaded functions for use with C